

Unified Project Plan: AI-Assisted Macroeconomic Modeling Dashboard

ECO 317 – Assignment #2

Project goal in plain language. Build one Streamlit app that solves three infinite-horizon stochastic macro models with Value Function Iteration, simulates them for at least 100 periods, produces exact policy-based forecasts, shows moments and graphs, uses polished academic styling, and is reliable enough to launch live with `streamlit run app.py` in front of the professor.

1. Project Objective and Assignment Requirements

1.1. What the finished project must do

1. Solve three models from class notes:
 - Model 1: stochastic consumption-savings with income shock.
 - Model 2: stochastic Robinson Crusoe model with capital and TFP shock.
 - Model 3: endogenous labor supply with wage shock.
2. Solve all three over an infinite horizon using Value Function Iteration (VFI).
3. Use a two-state Markov shock in every model.
4. Simulate each model for at least 100 periods.
5. Report mean, variance, lag-1 autocorrelation, and correlation with income or output.
6. Produce forecasts using solved policy functions, not trend fitting.
7. Provide a unified Streamlit dashboard with model switching, parameter sliders, and Markov probability controls.
8. Include a dark or navy interface with white chart areas and a Garamond-style serif look if possible.
9. Add dynamic written summaries built from live results using Python f-strings.
10. Be explainable by any teammate during the live demo.

1.2. What already exists in the project folder

This project already has the main folders and starter files. Use them instead of inventing a new structure.

```
ECO 317 Project 2/
|-- app.py
|-- config.py
|-- README.md
|-- requirements.txt
|-- assets/style.css
|-- models/
| |-- consumption_savings.py
| |-- robinson_crusoe.py
| |-- labor_supply.py
|-- solvers/vfi.py
|-- simulation/
| |-- simulate.py
| |-- forecast.py
| |-- moments.py
```

```
|-- utils/
| |-- grids.py
| |-- markov.py
| |-- interpolation.py
|-- tests/
```

Important repo rule. `requirements.txt`, `app.py`, `config.py`, `assets/style.css`, `models/`, `solvers/`, `simulation/`, `utils/`, and `tests/` already exist. Teammates should edit those files or create new files *inside those existing folders* when needed. Do not create a second top-level project layout.

1.3. How to think about the project

Simple mental model. The solver finds the optimal policy rules. The simulation uses those policy rules to generate time series. The forecast uses those same policy rules with a user-chosen future shock path. The Streamlit app is just the display layer on top of that pipeline.

2. Locked-In Model Specs

2.1. Shared rules for all three models

Item	Locked-in choice
Horizon	Infinite horizon
Solution method	Value Function Iteration
Shock process	Two-state Markov chain in each model
Grid size target	Usually 200–400 points for main state grids; keep defaults fast enough for live demo
Simulation target	At least 100 periods per model
Forecast rule	Use solved policy functions; do not fit separate trends
Moments	Mean, variance, lag-1 autocorrelation, correlation with income or output
Dashboard	One Streamlit app with model toggle, sliders, plots, tables, and written summaries
Runtime text	Deterministic Python f-strings based on computed results

Non-negotiable rule. If a textbook version differs from the course notes, the course notes win. The biggest grading risk is not a missing widget. It is coding the wrong economics.

2.2. Model 1: Stochastic consumption-savings

$$V(a_t, y_t) = \max_{a_{t+1}} \left\{ u(c_t) + \beta \sum_{y_{t+1}} P(y_{t+1} | y_t) V(a_{t+1}, y_{t+1}) \right\}$$

$$c_t + a_{t+1} = y_t + (1 + r_t)a_t, \quad c_t > 0$$

$$u(c) = \begin{cases} \frac{c^{1-\sigma} - 1}{1-\sigma}, & \sigma \neq 1 \\ \ln(c), & \sigma = 1 \end{cases}$$

Field	Decision
State variables	Current assets a_t and current income shock y_t
Control variable	Next-period assets a_{t+1}
Shock states	Low income and high income
Transition matrix	Two-state matrix; default project choice uses rows $[0.9, 0.1]$ and $[0.1, 0.9]$ unless the group changes it in the UI
Timing	Enter with a_t , observe y_t , choose a_{t+1} , back out c_t from the budget constraint
Solver outputs	Value function, policy indices, savings policy, consumption policy, diagnostics, state grid

2.3. Model 2: Stochastic Robinson Crusoe

$$V(k_t, z_t) = \max_{k_{t+1}} \left\{ u(c_t) + \beta \sum_{z_{t+1}} P(z_{t+1} | z_t) V(k_{t+1}, z_{t+1}) \right\}$$

$$c_t + k_{t+1} = y_t + (1 - \delta)k_t$$

$$y_{t+1} = z_{t+1}F(k_{t+1}), \quad F(k) = Ak^\alpha$$

Field	Decision
State variables	Current capital k_t and current TFP shock z_t
Control variable	Next-period capital k_{t+1}
Shock states	Low TFP and high TFP
Transition matrix	Two-state TFP Markov matrix; default project choice uses rows $[0.9, 0.1]$ and $[0.1, 0.9]$
Timing	Enter with installed capital k_t , observe z_t , choose k_{t+1} , then tomorrow's output follows $Y_{t+1} = F(K_{t+1})$ with the shock applied
Solver outputs	Value function, policy indices, capital policy, consumption policy, diagnostics, capital grid

Timing warning for Model 2. This is the most important model-specific caution in the whole project. Use the lecture timing literally: tomorrow's output is defined by $Y_{t+1} = F(K_{t+1})$. Do not silently replace it with a different textbook timing convention.

2.4. Model 3: Endogenous labor supply

$$V(a_t, w_t) = \max_{a_{t+1}, L_t \in [0,1]} \left\{ u(c_t) + v(1 - L_t) + \beta \sum_{w_{t+1}} P(w_{t+1} | w_t) V(a_{t+1}, w_{t+1}) \right\}$$

$$c_t + a_{t+1} = (1 + r_t)a_t + w_t L_t, \quad c_t > 0$$

Field	Decision
State variables	Current assets a_t and current wage shock w_t
Control variables	Next-period assets a_{t+1} and labor L_t
Shock states	Low wage and high wage
Transition matrix	Two-state wage Markov matrix; default project choice uses rows $[0.9, 0.1]$ and $[0.1, 0.9]$
Timing	Enter with assets, observe wage, choose labor and saving, then recover consumption and leisure $1 - L_t$
Solver outputs	Value function, savings policy, labor policy, consumption policy, diagnostics

Model 3 lock-in. The lecture-based lock-in sheet supports an asset-inclusive labor model with time endowment $T = 1$. Keep labor in $[0, 1]$, keep leisure as $1 - L$, and only drop the asset dimension if the instructor explicitly tells the group to use a simpler lecture version.

2.5. Cross-model coding convention

All three models should return the same top-level structure as much as possible:

- `value_function`
- `policy_indices`
- `policy_levels`
- `c_policy`
- `grid`
- `diagnostics`

3. Step-by-Step Build Plan

3.1. Step 1: Lock the equations before anyone writes serious code

Owner: Shared group work

Open these files first:

- the assignment PDF,
- `model_lockin_sheet.tex`,
- `execution_plan.tex`,
- `plan.tex`.

What to do:

- Confirm the Bellman equation, budget or resource constraint, state variables, controls, and timing for all three models.
- Write down any final naming choices for variables such as `a_grid`, `k_grid`, `labor_grid`, `P`, and `policy_idx`.
- Decide which quantities each model must simulate and plot.

Done when: every teammate can answer “what is the state, what is the control, what is the constraint, and what is the timing?” for all three models.

3.2. Step 2: Make sure the environment and root files are usable

Owner: Shared group work

Open or edit these files:

- `requirements.txt`

- `README.md`
- `app.py`
- `config.py`

What to do:

- Update `requirements.txt` if a needed package is missing. Do not recreate the file from scratch because it already exists.
- Keep `README.md` short and accurate so any teammate can install and launch the app.
- Use `app.py` as the entry point for the smoke test.
- Use `config.py` for default parameters shared across models.

Beginner note: if you are unsure whether something belongs in `app.py` or `config.py`, put only user interface code in `app.py` and put default model values in `config.py`.

Done when: `pip install -r requirements.txt` works and `streamlit run app.py` opens a basic page.

3.3. Step 3: Turn the existing folder tree into a clear skeleton

Owner: Shared group work

Use these existing folders:

- `models/`
- `solvers/`
- `simulation/`
- `utils/`
- `tests/`

What to do:

- Put model-specific economics in the matching file under `models/`.
- Put the generic VFI loop in `solvers/vfi.py`.
- Put time-series logic in `simulation/simulate.py`, forecast logic in `simulation/forecast.py`, and moments in `simulation/moments.py`.
- Put reusable numerical helpers in `utils/`.
- If you need a new helper file such as `utils/summaries.py`, create it inside the existing folder rather than making a new top-level folder.

Done when: the group has agreed where each type of code will live before the real implementation starts.

3.4. Step 4: Build the shared numerical utilities first

Owner: Aiden

Open or edit these files:

- `utils/grids.py`
- `utils/markov.py`
- `utils/interpolation.py`
- `simulation/moments.py`

What to do:

- In `utils/grids.py`, add helpers for linear grids, optional curved grids, clipping, and bounds checks.
- In `utils/markov.py`, add transition-matrix validation and shock-path simulation.
- In `utils/interpolation.py`, wrap `numpy.interp` so simulation and forecasting use one common interpolation rule.
- In `simulation/moments.py`, add mean, variance, lag-1 autocorrelation, and correlation helpers.

Done when: you can build a test grid, validate a 2-by-2 Markov matrix, simulate a simple shock path, and compute moments on a fake series.

3.5. Step 5: Build the generic VFI engine

Owner: Aiden

Open or edit this file:

- solvers/vfi.py

What to do:

- Write the main Bellman iteration loop.
- Let model files supply model-specific pieces such as utility, feasibility, state transition logic, and candidate controls.
- Track convergence error and store policy indices.
- Return diagnostics such as iteration count, final error, and converged flag.

```
V = np.zeros((n_grid, n_shocks))
for it in range(max_iter):
    V_new = np.empty_like(V)
    policy_idx = np.empty_like(V, dtype=int)
```

Done when: the solver converges on a tiny toy problem and returns a reusable result structure.

3.6. Step 6: Add feasibility checks before real model runs

Owner: Rida

Open or edit these files:

- models/consumption_savings.py
- models/robinson_crusoe.py
- models/labor_supply.py
- optionally utils/grids.py if shared clipping helpers help

What to do:

- For each model, define what counts as feasible.
- Reject any choice that makes consumption nonpositive.
- For Model 3, keep labor in $[0, 1]$ and keep leisure valid.
- Decide whether infeasible choices are filtered out or assigned `-np.inf`.

Beginner note: this step is mostly about preventing impossible economics from entering the maximization. If a control choice breaks the budget constraint, the code should reject it immediately.

Done when: extreme parameter choices fail cleanly or get filtered out instead of producing nonsense.

3.7. Step 7: Finish Model 1 completely and use it as the template

Owner: Rida

Open or edit these files:

- models/consumption_savings.py
- config.py
- tests/ for a basic Model 1 test file if needed

What to do:

- Build the asset grid and income states.
- Add the two-state Markov matrix.
- Implement CRRA utility with the log special case when $\sigma = 1$.

- Call the generic VFI engine and return savings and consumption policies.
- Keep the function output in the shared cross-model format.

```
asset_grid = np.linspace(a_min, a_max, n_a)
P = np.array([[0.9, 0.1],
              [0.1, 0.9]])
```

Done when: Model 1 produces smooth, believable savings and consumption policies and positive consumption everywhere.

3.8. Step 8: Add simulation and moments for Model 1

Owner: Lindsey

Open or edit these files:

- simulation/simulate.py
- simulation/moments.py
- models/consumption_savings.py if you need model-specific simulation helpers

What to do:

- Simulate a shock path for Model 1.
- Apply the saved policy functions period by period.
- Interpolate if the current asset level falls off-grid.
- Store assets, consumption, and income in arrays or a DataFrame.
- Compute the required moments.

Done when: the same random seed gives the same simulation and the moments match direct calculations from the simulated arrays.

3.9. Step 9: Add forecast logic for Model 1

Owner: Lindsey

Open or edit these files:

- simulation/forecast.py
- models/consumption_savings.py
- app.py later for UI hookup

What to do:

- Write a forecast function that takes the current state and a user-selected future shock path.
- Use solved policy functions only.
- Store forecast paths separately from the random simulation output.
- Keep the horizon configurable.

Done when: a chosen shock sequence like high, high, low visibly changes the forecast path in a model-consistent way.

3.10. Step 10: Implement Model 2 after a timing checkpoint

Owner: Ben

Open or edit these files:

- models/robinson_crusoe.py
- config.py
- tests/ for a Model 2 test file if needed

What to do:

- Re-check the notes and keep the $Y_{t+1} = F(K_{t+1})$ timing at the top of the file as a comment.

- Build the capital grid and TFP states.
- Add production, depreciation, and feasibility rules.
- Call the shared VFI engine.
- Return capital, consumption, and diagnostics in the common structure.

Done when: the model produces reasonable paths for capital, output, consumption, and investment, and the upper grid edge is not binding all the time.

3.11. Step 11: Implement Model 3 with labor-leisure trade-offs

Owner: Ben

Open or edit these files:

- `models/labor_supply.py`
- `config.py`
- `tests/` for a Model 3 test file if needed

What to do:

- Use the wage shock and time endowment $T = 1$.
- Create the labor grid.
- If using the lecture-backed asset-inclusive version, search jointly over labor and next-period assets.
- Recover labor, consumption, and savings policies.
- Add at least one static intuition graph idea, such as labor against wage or labor against the leisure-weight parameter.

Done when: labor stays in $[0, 1]$, leisure remains valid, and the resulting policy behavior is economically interpretable.

3.12. Step 12: Standardize the interface across all three models

Owner: Shared group work

Open or edit these files:

- `models/consumption_savings.py`
- `models/robinson_crusoe.py`
- `models/labor_supply.py`
- `simulation/simulate.py`
- `simulation/forecast.py`
- `simulation/moments.py`

What to do:

- Make sure all three models return similar key names.
- Use one shared simulation interface and one shared forecast interface where possible.
- Decide the key plotted series for each model.

Done when: switching from one model to another mostly changes the data, not the overall app logic.

3.13. Step 13: Build the Streamlit dashboard on top of working model code

Owner: Shared group work

Open or edit these files:

- `app.py`
- `config.py`

What to do:

- Add model navigation.

- Add sliders or number inputs for parameters and Markov probabilities.
 - Solve the selected model, run the simulation, run the forecast, and display results.
 - Keep `app.py` thin by calling functions from model, solver, simulation, and utility files.
- Done when:** a user can move between the three models without the app breaking.

3.14. Step 14: Add comparative statics and written summaries

Owner: Shared group work

Open or edit these files:

- `app.py`
- `simulation/moments.py`
- create `utils/summaries.py` inside `utils/` if you want summary logic in one place

What to do:

- Add at least one baseline-versus-alternative comparison.
- Compare two calibrations such as higher β , higher σ , or different shock persistence.
- Generate written summaries with deterministic f-strings based on computed moments.

Done when: the app text and comparison plots change when parameters change, and the wording matches the computed results.

3.15. Step 15: Add styling, caching, and performance protection

Owner: Shared group work

Open or edit these files:

- `assets/style.css`
- `app.py`

What to do:

- Put the dark or navy styling in `assets/style.css`.
- Use a serif fallback stack if exact Garamond is unavailable.
- Cache expensive solver calls in `app.py`.
- Choose default grid sizes and simulation lengths that still feel responsive during the demo.

Done when: the app looks intentional and standard slider changes update within a few seconds.

3.16. Step 16: Add tests and sanity checks

Owner: Shared group work

Open or create these files inside the existing tests/ folder:

- `tests/test_model1.py`
- `tests/test_model2.py`
- `tests/test_model3.py`
- `tests/test_moments.py`

What to do:

- Test positivity of consumption.
- Test policy bounds.
- Test convergence flags.
- Test that moments displayed in the app match direct calculations from the same data.
- Check for NaNs, infinities, and obviously jagged policies.

Done when: all three models pass a basic sanity suite and the dashboard outputs match the underlying numerical data.

3.17. Step 17: Keep a prompt log and comments the whole time

Owner: Shared group work

Create or edit these files if needed:

- create `prompt_log.md` in the project root
- add comments inside model, solver, simulation, and app files

What to do:

- For each major AI-assisted component, record the prompt, the file it affected, what was kept, and what was corrected.
- Add comments that explain the economics and numerical logic, not just the syntax.

Done when: if the professor points at a block of code, someone can explain what it does and how the AI-generated part was checked.

3.18. Step 18: Rehearse the live demo exactly as it will happen

Owner: Shared group work

What to practice:

- run `pip install -r requirements.txt`
- run `streamlit run app.py`
- switch across all three models
- move sliders and explain the economic effect
- answer code-location questions such as “where is the Bellman update?” or “where is interpolation used?”

Done when: any teammate can handle both a random economics question and a random code-workflow question without needing a rescue.

4. Team Ownership By Step

Step	Owner	Main responsibility
1–3	Shared	Lock equations, confirm environment, agree on file structure
4–5	Aiden	Shared utilities and generic VFI engine
6–7	Rida	Feasibility logic and full Model 1 implementation
8–9	Lindsey	Model 1 simulation, moments, and forecast logic
10–11	Ben	Model 2 and Model 3 implementation
12–18	Shared	Integration, dashboard, summaries, styling, testing, prompt log, and rehearsal

How to use this ownership table. It does not mean one person works alone forever. It means that person is the first owner of the step, writes the first clean version, and makes sure the rest of the group understands it before the team moves on.

4.1. Where each member’s work lives in the final project

Use this box during the presentation to quickly point at the file that contains your contribution. If the professor asks “who wrote this and where is it?” every teammate should be able to answer immediately.

Aiden – Steps 4–5: Shared utilities and generic VFI engine.

- `utils/grids.py` – `linear_grid` (line 11: evenly spaced grid via `np.linspace`), `curved_grid` (line 16: quadratic spacing denser near lower bound), `clip_to_grid` (line 22: clamps values to grid bounds), `is_consumption_positive` (line 27: checks $c > 0$ for all elements), `is_labor_feasible` (line 32: checks $L \in [0, 1]$).
- `utils/markov.py` – `validate_transition_matrix` (line 11: verifies square, non-negative, rows sum to 1), `simulate_shock_path` (line 26: draws a length- T index path from the Markov chain using `np.random.default_rng`).
- `utils/interpolation.py` – `interp_policy` (line 12: wraps `numpy.interp` for off-grid policy evaluation; clamped at boundaries).
- `simulation/moments.py` – `compute_moments` (line 20: returns dict of mean, variance, lag-1 autocorrelation, and correlation with income/output for every simulated series).
- `solvers/vfi.py` – `iterate(V_init, bellman_operator)` (line 23: callback-style VFI engine; loops until $\max |V_{n+1} - V_n| < \text{tol}$ or `max_iter`; returns `value_function`, `policy_indices`, and `diagnostics` dict with `iterations`, `final_error`, `converged`).

Original prototype: Other member's work/`eco 317 project 2 steps 4-5.py`. Ported into the project modules above, cleaned of REPL prompts, and adapted to the project's callback-style solver API.

Rida – Steps 6–7: Feasibility checks and full Model 1 implementation.

- `models/consumption_savings.py`:
 - `utility` (line 30): CRRA utility $u(c) = (c^{1-\sigma} - 1)/(1 - \sigma)$ with $\log(c)$ when $\sigma = 1$; returns $-\infty$ where $c \leq 0$.
 - `build_grids` (line 44): creates asset grid and income-shock array from params.
 - `feasibility_mask` (line 51): returns boolean array shape (n_a, n_a, n_y) marking (a, a', y) triples where $c > 0$.
 - `solve` (line 72): main entry point. Pre-computes the utility matrix $u(c)$ for every (a_i, a'_k, y_j) triple, builds a Bellman closure (line 115) that computes $EV = V \cdot P^T$ and maximises over a' , calls `solvers.vfi.iterate`, then extracts savings policy $a'(a, y)$ and consumption policy $c(a, y)$ from the budget constraint.
- `tests/test_model1.py` – convergence, consumption positivity, no-NaN, shape consistency, policy bounds.

Original submission: Other member's work/Rida's code/feasibility checks (Bash smoke-test script). Converted into Python `pytest` tests and used as acceptance criteria for the Model 1 solve.

Lindsey – Steps 8–9: Simulation and forecast logic.

- `simulation/simulate.py`:
 - `simulate_model` (line 19): main dispatch function; calls model-specific helpers below.
 - `_simulate_model1` (line 80): draws a Markov shock path via `utils.markov.simulate_shock_path`, then for each period t : reads shock s_t , looks up y_t , interpolates c_t and a_{t+1} from the solved policies using `utils.interpolation.interp_policy`. Returns dict with keys `a`, `c`, `y`, `shock_idx`.
 - `_simulate_model2` (line 104): same pattern for the Robinson Crusoe model; also computes $Y_t = z_t F(k_t)$ and investment $I_t = k_{t+1} - (1 - \delta)k_t$.
 - `_simulate_model3_assets` (line 140): simulates the asset-inclusive labor model; interpolates savings, labor, and consumption policies on the asset grid.
 - `_simulate_model3_labor_only` (line 171): direct index lookup (no interpolation needed since there is no continuous state).
- `simulation/forecast.py`:
 - `forecast_model` (line 15): main dispatch; takes a user-chosen shock sequence and current state.
 - `_forecast_model1` (line 64): deterministic forward iteration using the same interpolated policies as simulation but with the user-specified shock path instead of random

5. Final Demo and Validation Checklist

5.1. Definition of done

1. The equations match the course notes.
2. All three models solve over the infinite horizon with VFI.
3. Each model uses a two-state Markov shock.
4. Each model simulates at least 100 periods.
5. Forecasts come from solved policy functions.
6. The app shows graphs, moments, and dynamic summaries.
7. The dashboard supports parameter sliders and smooth model switching.
8. The styling is presentation-ready.
9. The code is commented and the prompt log exists.
10. The project runs from scratch on the laptop used for the demo.

5.2. Live-demo checklist

- `pip install -r requirements.txt` works cleanly.
- `streamlit run app.py` launches without manual fixes.
- Model 1, Model 2, and Model 3 all load.
- The slider changes are responsive enough for a live walkthrough.
- Forecast controls visibly change the forecast path.
- Comparative statics visibly change at least one policy or time-series graph.
- Moments displayed in the app are numerically correct.
- The team can explain Bellman logic, feasibility checks, Markov shocks, interpolation, caching, and AI verification.

Bottom line. The best version of this assignment is the one where the economics are locked in first, the numerical methods are trustworthy second, and the dashboard polish comes third. If the group can launch the app from scratch, change parameters live, and explain exactly why the code is economically correct, the project is in strong shape.